Addressing data. Spark, Databricks, and other analytics tools reach ADLS Gen2 through the ABFS driver, using an address of this form: **abfss://<container>@<storageaccount>.dfs.core.windows.net/<path>**. The '**s'** in **abfss** means the connection always uses encryption. The dfs endpoint is the Data Lake endpoint, while the blob endpoint is the ordinary Blob one, and a hierarchical account can be reached through both.

```
# Create a storage account with the hierarchical namespace turned on
az storage account create \
   --name stdatalakedev001 \
   --resource-group rg-tricloud-dev \
   --location centralindia \
   --sku Standard_LRS \
   --kind StorageV2 \
   --enable-hierarchical-namespace true \
   --allow-blob-public-access false
# Create containers (file systems) and a directory inside one
# Give a person the right to read and write data in the account
# Address a file from Spark or Databricks
```

- **Decide at creation.** Choose hierarchical namespace when the storage account is created. Turning it on later for an existing account is possible only through a special upgrade process with limitations, so it is best planned from the start for analytics data.
- **Plan the layout.** A common design uses one container per stage, such as bronze, silver, and gold (the medallion layers from the Databricks sessions), with folders by source and by date inside, for example sales/2026/10/.
- **Avoid many tiny files.** Large numbers of very small files slow analytics down, so data is usually written in fewer, larger files.
- **Link to Databricks.** In Databricks, a storage credential and an external location (Unity Catalog documentation) are how a workspace is given governed access to exactly this kind of container.

Keep it private. Turn off public blob access and grant access through roles, so data is never open to the internet by accident.

SESSION 2 — AZURE I: IDENTITY AND ACCESS MANAGEMENT

- **Topic:** Azure I — Identity and Access Management
- **Date:** 22nd October 2026

4. Microsoft Entra ID Fundamentals

Microsoft Entra ID is the identity service for Microsoft's cloud. It keeps the list of people and applications that exist in your organization and checks that someone really is who they claim to be when they sign in. It was formerly named **Azure Active Directory (Azure AD)**.

Authentication and authorization. Authentication answers who are you, and is proven with a password, a code on a phone, or a fingerprint. Authorization answers what are you allowed to do, Entra ID handles authentication, and Azure RBAC handles authorization of Azure resources.

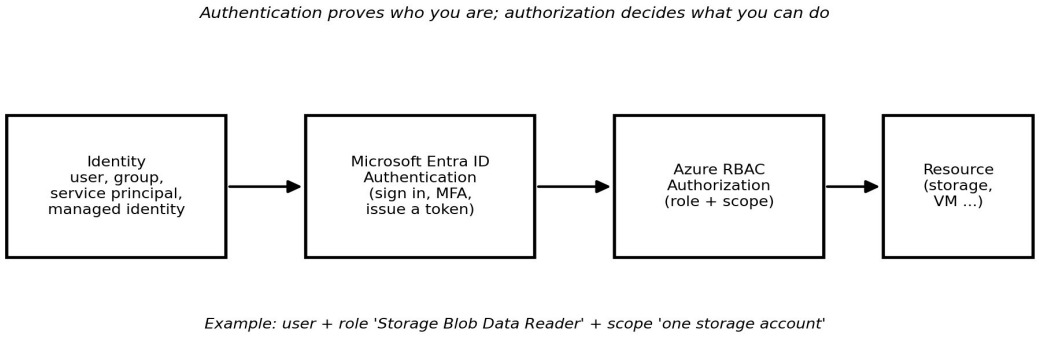


Diagram 4: An identity signs in through Entra ID, and Azure RBAC then checks its role and scope before allowing access.

Term	**Simple meaning**
User	A person's account in the directory, for example an employee.
Group	A collection of users. Giving access to a group is easier and safer to manage than giving it to people one at a time.
Service principal	An identity for an application or automation tool, so software can sign in without using a person's account.
Managed identity	A special service principal that Azure creates and manages for a resource, with no password for anyone to store. System-assigned ones belong to one resource, and user-assigned ones can be shared by several.
App registration	The record that introduces an application to Entra ID so that it can sign in or be signed in to.
Role	A named set of permissions, such as Reader (view only), Contributor (create and change but not grant access), or Owner (everything, including granting access).
Scope	The level at which a role applies: a management group, a subscription, a resource group, or one resource. A role given at a higher scope is inherited by everything below it.
Role assignment	The link that says who gets which role at which scope. It has exactly three parts: the identity, the role, and the scope.

Two kinds of roles. Entra roles control the directory itself, such as creating users or resetting passwords, and examples are Global Administrator and User Administrator. Azure roles control Azure resources such as storage accounts and virtual machines, and examples are Owner, Contributor, and Reader. They are separate systems, so being an administrator of the directory does not automatically give access to the data in a storage account.

Security feature	What it does for you
Multifactor authentication (MFA)	Asks for a second proof, such as a phone prompt, so a stolen password alone is not enough.
Single sign-on (SSO)	One sign-in gives access to many applications.
Conditional Access	Rules such as require MFA when signing in from a new location. Available in the paid tiers.
Privileged Identity Management	Gives high-powered roles only for a limited time and only when needed. Available in the highest paid tier.

- **Give roles to groups.** Add people to a group and give the group the role, so that access is changed by editing membership and not by touching every resource.
- **Least privilege.** Give the smallest role at the smallest scope that does the job. Reader on one resource group is safer than Owner on a whole subscription.
- **Prefer identities over passwords.** Applications should use managed identities or service principals and not a person's password or a stored key. Azure Databricks uses this idea when it reaches storage through a managed identity.
- **Tie it to storage.** To read a container in ADLS Gen2, an identity needs two things: to be able to sign in through Entra ID, and a Storage Blob Data role on that account, container, or folder.

5. Quick Revision: Key Terms

Term	One-line meaning
Cloud computing	Renting computing, storage, and software over the internet and paying for what you use.
Region / availability zone	A geographic area of data centers / a separate data center location inside a region.
Subscription	The billing and access boundary under which Azure resources are created.
Resource group	A folder of related resources that share one lifecycle.
Storage account	The account that groups Azure's storage services and their shared settings.
Container (file system)	The top-level box inside a storage account that holds folders and files.
Hierarchical namespace	The feature that gives storage real folders, turning Blob storage into ADLS Gen2.
Microsoft Entra ID	Microsoft's identity service for signing in and managing users and applications.

RBAC	Role-based access control: giving access by assigning a role to an identity at a scope.
Managed identity	An identity Azure manages for a resource, so there is no password to store.

6. Creating Managed Identities

A managed identity is an identity that Azure creates and looks after for a resource, so that the resource can sign in to other Azure services without any password, key, or secret in your code. When the resource is deleted, a system-assigned identity is deleted with it.

Type	**How it works**
System-assigned	Turned on directly on one resource, such as a virtual machine or a web app. It is tied to that resource's life: it is created when you enable it and removed when the resource is deleted. It cannot be shared. Best when one resource needs its own access.
User-assigned	Created on its own as a separate Azure resource, then attached to one or more resources. It lives independently, so it survives if a resource is deleted, and the same identity (and the same permissions) can be shared by several resources. Best when many resources need the same access, or when the identity must be created and given permissions before the resource exists.

Steps to create and use one. The portal and the CLI follow the same four steps:

1. create (or enable) the identity,
2. find its principal ID,
3. give that ID a role on the target resource, and
4. then let the application use it.

- **In the portal.** For a system-assigned identity, open the resource, choose Identity, and switch Status to On. For a user-assigned one, search for Managed Identities, choose Create, then attach it from the Identity page of each resource.
- **An identity has no power yet.** Creating it only gives the resource an identity. It can do nothing until it is given a role, which is the next two sections.
- **No secrets to rotate.** Azure handles the credentials and renews them automatically, which removes a whole class of leaked-password problems.
- **Link to Databricks.** Azure Databricks uses a managed identity, through an access connector, so that Unity Catalog can reach a storage account without any stored key. That identity is given the Storage Blob Data Contributor role.

7. Azure RBAC Roles and Scope

Azure role-based access control (RBAC) is the system that decides what an identity may do to Azure resources. Instead of giving each person a custom list of permissions, you give them a role, which is a ready-made bundle of permissions with a clear name, such as Reader or Contributor. The role says what is allowed, and the scope says where it is allowed.

Built-in role	**What it allows**
Owner	Full control of the resources at the scope, including the right to give other people access.
Contributor	Create and manage resources, but not give other people access.
Reader	View resources only, with no changes.
User Access Administrator	Manage who has access to the resources, but not the resources themselves.
Storage Blob Data Reader	Read and list blobs, files, and folders (data access).
Storage Blob Data Contributor	Read, write, and delete blobs and containers (data access).
Storage Blob Data Owner	Full control of blob data, including setting folder permissions (ACLs) in a hierarchical account.

Management actions vs. data actions. Azure separates controlling a resource from using the data inside it. Management actions create, configure, or delete the storage account itself, and data actions read or write the files inside it. Roles such as Owner and Contributor cover management actions, and the Storage Blob Data roles cover data actions. This is why a person who is Owner of a storage account can still be refused when they try to read a file: they have not been given a data role.

Scope. Scope is the boundary a role applies to. In Azure there are four levels: management group, subscription, resource group, and resource. A role given at one level is inherited by everything below it, so Reader on a subscription means read access to every resource group and resource inside it.

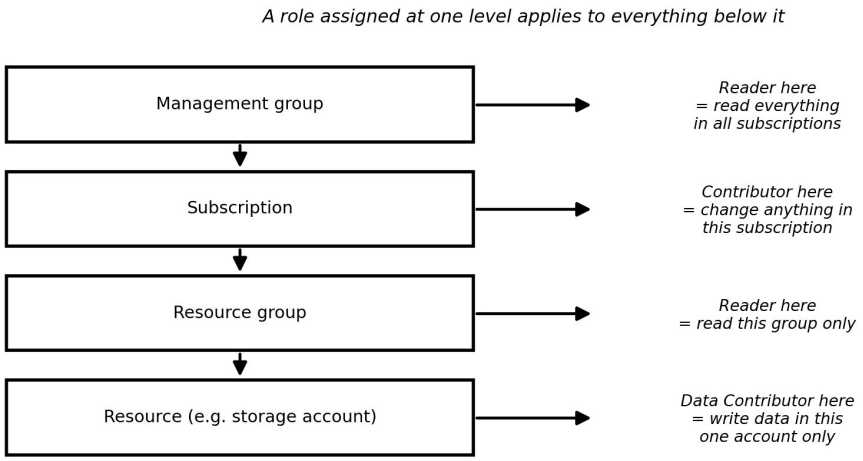


Diagram 5: A role assigned at one level flows down to everything beneath it.

- **Permissions add up.** If an identity has several roles that apply, it gets the combined permissions of all of them. There is no need to repeat a permission, and no ordinary role takes a permission away.
- **Smallest scope that works.** Prefer a resource group or a single resource to a whole subscription. The wider the scope, the more damage a mistake or a stolen identity can do.
- **Custom roles.** If no built-in role fits, an organization can define its own role by listing the exact actions it allows, but it is best to use a built-in role wherever one is suitable.
- **Changes take a few minutes.** A new role assignment can take several minutes to take effect everywhere, so a request that fails right after granting a role often works a few minutes later.

8. Principal, Role, and Resource Assignments

A role does nothing on its own. It becomes real only when it is assigned, and every assignment answers three questions: who is receiving it, which role is being given, and where it applies.

These three parts are the security principal, the role definition, and the scope.

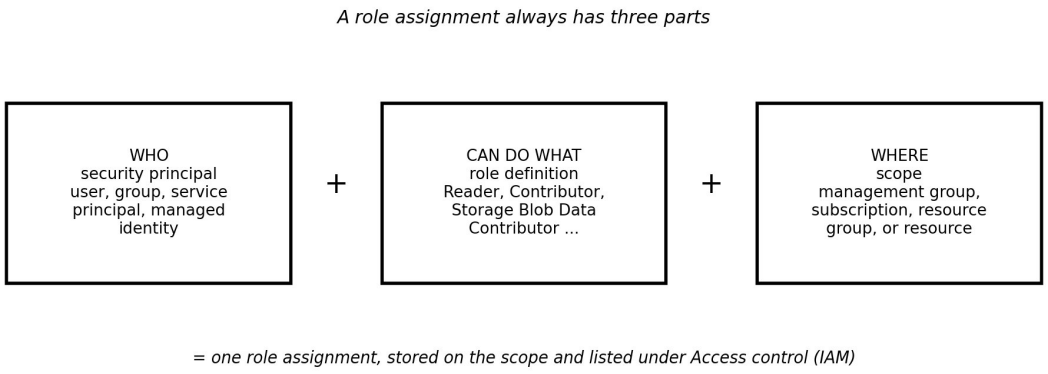


Diagram 6: Every role assignment is exactly one principal, one role, and one scope.

Part	Simple meaning
Security principal	The identity receiving the access. It can be a user, a group, a service principal, or a managed identity. It is referred to by its object ID.
Role definition	The named set of permissions being given, such as Reader or Storage Blob Data Contributor.
Scope	The level the access applies to: a management group, a subscription, a resource group, or one resource.
Role assignment	The record that joins the three. It is stored at the scope, and can be seen under Access control (IAM) on that resource, group, or subscription.

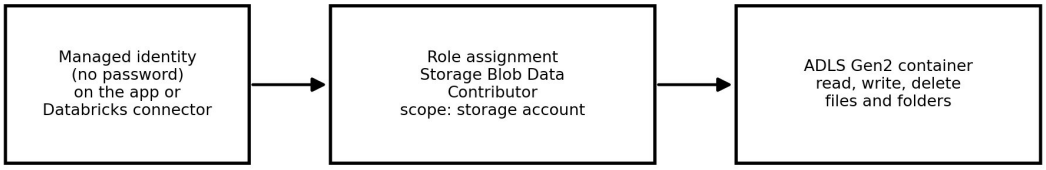
Resource assignments. There are two ideas here and it helps to keep them apart. One is giving an identity a role at the scope of a single resource, for example a role on one storage account, which is the most precise form of access. The other is assigning an identity to a resource, for example attaching a managed identity to a virtual machine (section 7), so that the resource can act with that identity. A typical setup uses both: assign the identity to the resource, then give that identity a role on the resource it needs.

- **Who can give access.** Only someone with permission to write role assignments can create one, which in practice means Owner or User Access Administrator at that scope. A Contributor can build resources but cannot grant access to them.
- **State the principal type.** When assigning to a newly created identity, such as a managed identity, add **--assignee-principal-type**. Azure then does not have to wait for the new identity to appear in the directory, which avoids a common error right after creation.
- **Check who has access.** Use the Access control (IAM) page of a resource, with Role assignments to see everything given, and Check access to see what one identity can do.
- **Time-limited access.** With Privileged Identity Management, a person can be eligible for a powerful role and activate it only when needed, instead of holding it all the time.

9. Practice: The Storage Blob Data Contributor Pattern

The pattern. A very common task in data work is letting an application, a pipeline, or a group of engineers read and write the files in a storage account. The standard solution is to give an identity the Storage Blob Data Contributor role on the storage account (or on one container). The identity is usually a managed identity for software, or a security group for people. This section puts sections 6 to 9 together in one exercise.

The Storage Blob Data Contributor pattern



Give the role to a group or a managed identity, never to a shared key or an individual's password

Diagram 7: A managed identity, a role assignment, and a container: the three parts of the pattern.

Step	What to do
1. Prepare	Use the resource group and the ADLS Gen2 storage account from Session 1, such as stdatalakedev001 with a container named bronze.
2. Create the identity	Create a user-assigned managed identity, id-tricloud-data (section 7), and note its principal ID.
3. Choose the scope	Decide how wide the access should be. The storage account lets the identity reach every container in it, and a single container is narrower. Use the smallest scope that does the job.
4. Assign the role	Give the identity the Storage Blob Data Contributor role at that scope.

5. Test it works	Upload and list a file while signed in with the identity or as a test user holding the same role, using Entra ID sign-in and not an account key.
6. Test the limit	Confirm that an identity with only Reader, or with no data role, is refused when it tries to write.
Symptom	**Likely cause and fix**
403 error: not authorized to perform this operation (permission mismatch)	The identity has no data role, or only a management role such as Owner or Contributor. Add a Storage Blob Data role.
The role was just assigned but access is still refused	Role assignments can take several minutes to apply. Wait, then try again, signing in again if needed.
Works with an account key but not with --auth-mode login	The key bypasses roles. The Entra ID path needs the role, so a missing role shows up only when using login.
The identity can read but not write	It has Storage Blob Data Reader. Change the assignment to Contributor, or add it.
Access is refused from another network	The storage account's firewall may be limiting networks. Check the Networking settings and allow the client's network or use a private connection.

- **Roles go to groups and managed identities.** Assign the role to a security group of engineers or to a managed identity, and not to individuals, so access is managed by membership and not by editing each resource.
- **No keys in code.** A storage account key gives access to everything in the account and cannot be limited by role. Using a managed identity with this role gives least-privilege access and nothing to leak.
- **What Contributor does not do.** Storage Blob Data Contributor reads, writes, and deletes data, but does not set folder-level permissions (ACLs) in a hierarchical account. That needs Storage Blob Data Owner, or ACL rights on the specific folder.
- **The same pattern in Databricks.** Unity Catalog's storage credential uses the managed identity of an Azure Databricks access connector, and that identity is given Storage Blob Data Contributor on the storage account. This is the access path behind the external locations in the Databricks documentation.

- **Clean up.** When the exercise is finished, delete the test files and the role assignment, or delete the resource group, so nothing keeps running or stays accessible.